PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad frontend con React, React Router Dom y manejo de API REST

Presentado a: Yoly Alexandra Guevara Muñoz

Por Aprendiz: Miguel Angel Rivera

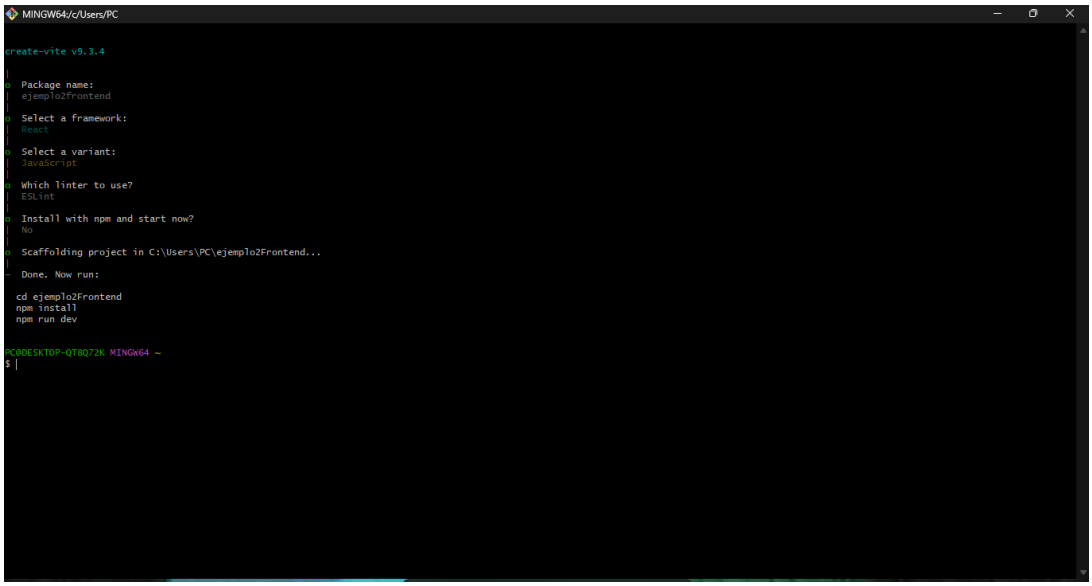
Ficha: 3312932

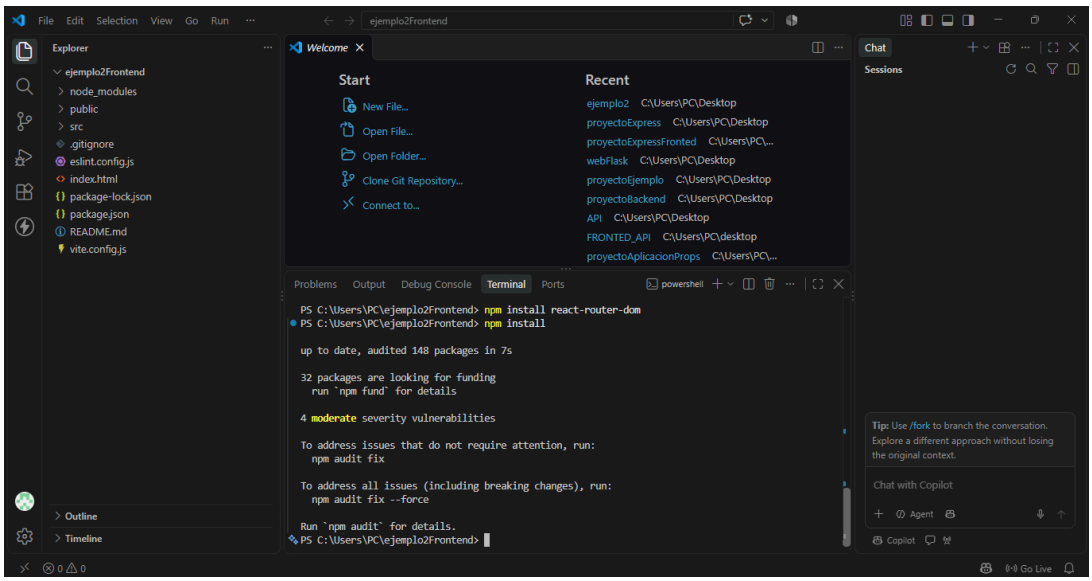
Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

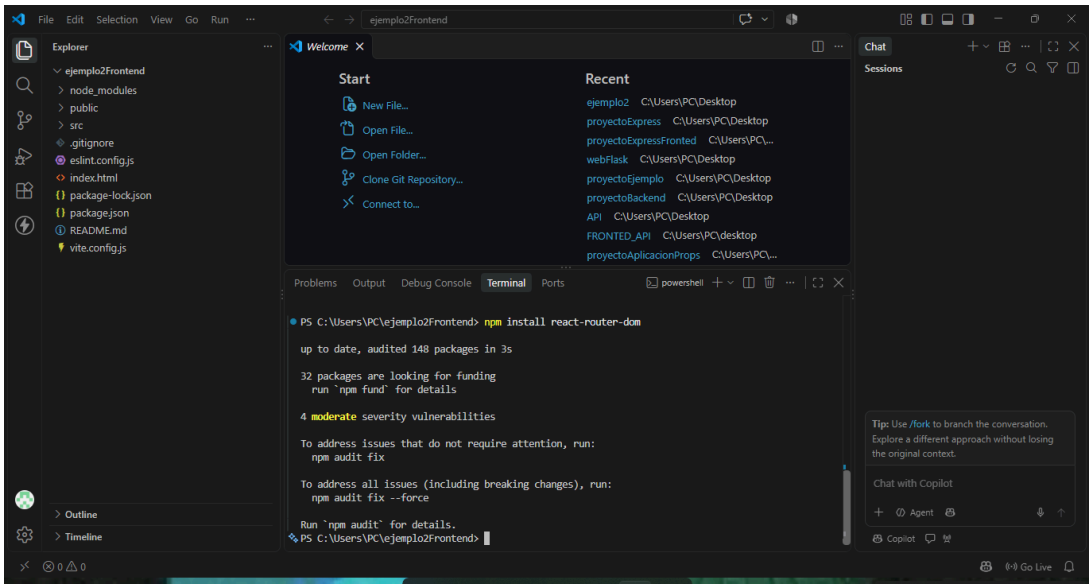
Popayán, día 01 de mes 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Fase 1:







Investigación:

- **React:** librería de JavaScript para construir interfaces con componentes reutilizables.
- **React Router DOM:** librería que permite navegar entre páginas sin recargar el navegador.
- **Link vs useNavigate():** Link es un enlace que el usuario pulsa. useNavigate() navega desde el código, por ejemplo después de un clic en un botón o al terminar una acción.
- **BrowserRouter, Routes, Route:** BrowserRouter activa la navegación, Routes agrupa las rutas y cada Route une una dirección con un componente.
- **useState:** guarda un valor que, al cambiar, vuelve a dibujar el componente. En un formulario, cada input guarda su valor con value y onChange.
- **fetch():** hace peticiones HTTP al backend y devuelve la respuesta.
- **Frontend, backend y base de datos:** el frontend muestra la interfaz y envía peticiones, el backend las recibe y consulta la base de datos, y la base de datos guarda los datos.

Fase 6:

Error	Causas y consecuencias	Corrección	Prueba

onClick={navigate("/dashboard")}	Se ejecuta al cargar, no al hacer clic	onClick={() => navigate("/dashboard")}	Hacer clic en el botón
useNavigate() fuera de BrowserRouter	Error en pantalla	Envolver App con BrowserRouter en main.jsx	Abrir la app
path mal escrito en una Route	La ruta no muestra nada	Que el path coincida con el Link	Entrar a la ruta directo
URL incorrecta en fetch (puerto o ruta)	No llega al backend	Usar http://localhost:5000/personas	Ver la petición en la pestaña Network
Falta Content-Type	Express no lee el body	Agregar headers: {"Content-Type": "application/json"}	Que se guarde el registro
Falta JSON.stringify	Se envía [object Object]	body: JSON.stringify({...})	Que se guarde el registro
Error de CORS	El navegador bloquea la petición	app.use(cors()) en Express	Sin errores rojos en la consola
Datos incompletos o inválidos	Se guardan registros vacíos o incorrectos	Validar antes del fetch	Enviar vacío y ver el mensaje
Registros que no se actualizan	La tabla queda desactualizada	Llamar consultar() tras guardar	Guardar y ver la fila nueva
Backend apagado	La app falla sin avisar	.catch con mensaje de error	Apagar la API y guardar

preguntas de análisis:

Ruta frontend vs backend: la del frontend muestra una pantalla; la del backend recibe peticiones y devuelve datos.

React Router DOM: permite navegar entre páginas sin recargar.

Link vs useNavigate(): Link es un enlace visible; useNavigate() navega desde el código.
El frontend no se conecta a MySQL: expondría las credenciales y la base de datos a cualquiera. El backend es el intermediario seguro.
fetch(): envía peticiones HTTP al backend y recibe la respuesta.
JSON.stringify(): convierte el objeto de JavaScript en texto JSON, que es lo que viaja por HTTP.
URL incorrecta: fetch falla, o el servidor no encuentra la ruta, y no llegan datos.
Error de CORS: pasa cuando el backend no permite el origen del frontend, porque están en puertos distintos.
Actualizar la tabla: después de guardar, se vuelve a llamar la consulta y setPersonas redibuja la tabla.
Validar en frontend y backend: el frontend da respuesta rápida al usuario, el backend protege la base de datos, porque el frontend se puede saltar.
Comprobar que se guardó en MySQL: con SELECT * FROM persona; en Workbench.
Navegar y proteger una ruta: navegar es ir a una página; proteger es impedir el acceso sin autenticación.